

UNIVERSITY BILINGUAL INTERNATIONAL AND SUSTAINABLE

REPORT

"PRACTICE 1: MULTI-SOURCE PROFILING AND INGESTION IN AWS S3 WITH PYTHON"

EXTRACTION KNOWLEDGE

PRESENTS:

ANA PAOLA IRIGOYEN JUÁREZ

23-02-004

TEACHER

JESUS LOBATO BAEZ

SAN JOSE CHIAPA, PUEBLA, PUE. OCTOBER, 18th, 2026.

Reporte Técnico de la Práctica 1

1. Introducción y Configuración

1.1 Objetivo

Esta práctica tiene como propósito conectar un script de Python a un bucket de **Amazon S3** mediante **boto3**, descargar en memoria dos archivos de distinta naturaleza, uno estructurado (**.csv**) y uno semiestructurado (**.json**), y generar con **pandas** un perfilamiento básico de sus tipos de datos y estadísticos descriptivos. La descarga se hace directamente a memoria con `io.BytesIO`, sin guardar los objetos en el disco local.

1.2 Arquitectura de AWS S3 utilizada

Amazon S3 es un servicio de almacenamiento de objetos: los datos se guardan como *objetos* dentro de *buckets* y cada objeto se identifica por una clave (*key*). El prefijo `raw/` funciona como una "carpeta" lógica y representa la zona de datos crudos, es decir, datos tal como llegan de la fuente y sin transformar, que es el primer nivel de un flujo de ingesta típico. Para esta práctica se usó el bucket `aws-academy-datacenter-bucket-23-02-004`, en la región `us-east-1`, con los dos objetos que se muestran en la tabla 1.

Clave (key)	Formato	Tipo de dato	Tamaño	Última modificación
raw/transacciones.csv	CSV	Estructurado (tabular)	39.8 KB	30 sep 2026, 2:20 PM
raw/logs_m2m.json	JSON	Semiestructurado (registros anidados)	89.9 KB	18 oct 2026, 3:08 PM

Tabla 1. Objetos almacenados en el prefijo raw/ del bucket (datos tomados de la consola de S3).

El script se autentica con las credenciales temporales de AWS Academy, solicita cada objeto con `get_object`, lee el cuerpo de la respuesta como bytes en memoria y los entrega a pandas, que construye un DataFrame sobre el cual se calcula el perfilamiento. La figura 1 resume este flujo.

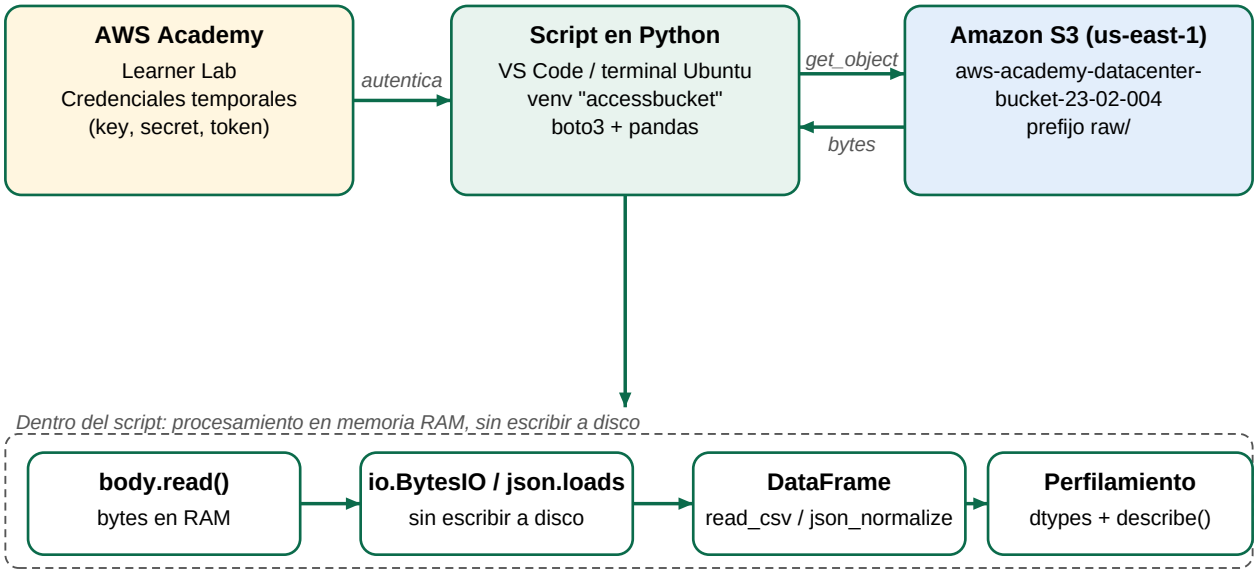


Figura 1. Arquitectura y flujo de datos de la práctica.

1.3 Preparación del entorno de desarrollo

El script se ejecutó dentro de un entorno virtual de Python llamado `accessbucket`, de modo que las dependencias queden aisladas del resto del sistema. Los pasos fueron:

```
python3 -m venv accessbucket          # crear el entorno virtual
source accessbucket/bin/activate        # activarlo (el prompt muestra "(accessbucket)")
pip install boto3 pandas                # instalar las dependencias
pip freeze > requirements.txt           # registrar versiones para el repositorio
```

Las librerías necesarias son **boto3**, el SDK oficial de AWS para Python, y **pandas**, que aporta los DataFrames y las funciones de perfilamiento (`dtypes`, `describe`, `json_normalize`).

1.4 Gestión de credenciales de AWS Academy

AWS Academy (Learner Lab) no entrega credenciales permanentes: genera un conjunto **temporal** formado por *Access Key ID*, *Secret Access Key* y *Session Token*, que deja de funcionar cuando termina la sesión del laboratorio. Boto3 puede leerlas de tres maneras: del archivo `~/.aws/credentials`, de las variables de entorno `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY` y `AWS_SESSION_TOKEN`, o como parámetros explícitos de `boto3.client`. En las pruebas documentadas en este reporte se pasaron como parámetros del cliente, lo cual es cómodo en el laboratorio pero **no es aceptable en un repositorio**, porque cualquier persona con acceso al código vería las claves.

Por eso la versión del script que se entrega en Git **no contiene ninguna credencial**: crea el cliente sin claves y deja que boto3 las resuelva con su cadena de búsqueda predeterminada (variables de entorno o `~/.aws/credentials`). Además, el archivo `.gitignore` excluye los archivos de credenciales, `.env` y el entorno virtual.

```
# Versión para el repositorio: sin credenciales en el código
s3_client = boto3.client('s3', region_name='us-east-1')

# Credenciales temporales en la sesión (se reemplazan en cada sesión de AWS Academy):
# export AWS_ACCESS_KEY_ID=<access key>
# export AWS_SECRET_ACCESS_KEY=<secret key>
# export AWS_SESSION_TOKEN=<session token>
```

2. Desarrollo Paso a Paso

2.1 Ejecución del script y generación del JSON

El script se lanzó con `python3 procesar_y_perfilar_s3.py`. El bloque principal ejecuta tres acciones en orden: (1) genera y sube a S3 el archivo `raw/logs_m2m.json`, (2) perfila `raw/transacciones.csv` y (3) perfila `raw/logs_m2m.json`. El primer paso fue necesario porque el objeto JSON no existía en el bucket, así que se crearon **300 registros sintéticos de comunicación máquina a máquina (M2M)** con una semilla fija (`random.seed(101)`). Cada registro describe un evento de un dispositivo e incluye un bloque anidado de métricas:

Campo	Contenido generado
log_id, timestamp	Identificador (LOG-10001 en adelante) y fecha-hora ISO 8601 dentro de los últimos 7 días
device_id, ip_address	Uno de 20 dispositivos (DEV-M2M-001 a 020) y una IP 192.168.1.x
level, event	Nivel (INFO, WARNING, ERROR, SUCCESS) y tipo de evento (DEVICE_HEARTBEAT, DATA_SYNC, CONNECTION_FAILED, FIRMWARE_UPDATE, SENSOR_READ)
metrics (anidado)	cpu_usage_pct (5 a 99), latency_ms (10 a 450) y bytes_sent (128 a 65,536)

Tabla 2. Estructura de cada registro del archivo logs_m2m.json.

El mensaje "Subiendo 300 logs M2M..." seguido de "¡Archivo JSON generado y subido correctamente!" (figura 2) confirma que `put_object` terminó sin errores.

2.2 Ingesta y perfilamiento del archivo CSV

```
(accessbucket) ubuntu@ip-172-31-72-15:~/accessbucket/accessbucket$ python3 procesar_y_perfilar_s3.py
=== Generando datos sintéticos para raw/logs_m2m.json ===
Subiendo 300 logs M2M a s3://aws-academy-datacenter-bucket-23-02-004/raw/logs_m2m.json...
¡Archivo JSON generado y subido correctamente!

=== Procesando objeto S3: raw/transacciones.csv ===
Dimensiones del dataset: 500 filas x 7 columnas

Perfil de Tipos de Datos e Identificación de Tipos:
id_transaccion        str
id_usuario            str
fecha                 str
monto             float64
metodo_pago           str
estado                str
ciudad                str
dtype: object

Resumen de Estadísticos Descriptivos (Cuantitativos):
       count        mean         std    min      25%      50%      75%      max
monto  500.0  1244.88934  716.530856  17.16  627.115  1204.46  1805.785  2499.94
------------------------------------------------------------
```

Figura 2. Salida del script: subida del JSON y perfilamiento de raw/transacciones.csv.

Interpretación. El CSV se descargó y se cargó en un DataFrame de **500 filas por 7 columnas**. Pandas identificó la columna `monto` como numérica (`float64`) y las otras seis como texto (en la salida aparecen como `str`, y el tipo del contenedor es `object`). La tabla 3 interpreta el papel de cada columna.

Columna	Tipo detectado	Interpretación
id_transaccion	str	Identificador único de la transacción
id_usuario	str	Identificador del usuario que la realizó
fecha	str	Fecha de la transacción; llegó como texto y conviene convertirla a datetime
monto	float64	Importe de la transacción (única variable cuantitativa)
metodo_pago	str	Variable categórica (forma de pago)
estado	str	Variable categórica (estado de la transacción)
ciudad	str	Variable categórica (ubicación)

Tabla 3. Perfil de tipos de datos del CSV.

count	mean	std	min	25%	50%	75%	max
500	1,244.89	716.53	17.16	627.12	1,204.46	1,805.79	2,499.94

Tabla 4. Estadísticos descriptivos de la columna monto (valores redondeados a dos decimales).

Como el *count* es 500, igual al número de filas, la columna `monto` **no tiene valores nulos**. La media (1,244.89) y la mediana (1,204.46) son muy cercanas, lo que indica una distribución prácticamente simétrica, sin valores extremos que jalen el promedio. Los montos van de 17.16 a 2,499.94 y el rango intercuartílico (627.12 a 1,805.79) cubre la mitad central de las transacciones. Además, una

desviación estándar de 716.53 es cercana a la que tendría una distribución uniforme entre 0 y 2,500 (alrededor de 722), lo cual sugiere que los montos están repartidos de forma pareja en todo ese intervalo.

2.3 Ingesta y perfilamiento del archivo JSON

```
=== Procesando objeto S3: raw/logs_m2m.json ===
Dimensiones del dataset: 300 filas x 9 columnas

Perfil de Tipos de Datos e Identificación de Tipos:
log_id                    str
timestamp                 str
device_id                 str
ip_address                str
level                     str
event                     str
metrics.cpu_usage_pct     float64
metrics.latency_ms        int64
metrics.bytes_sent        int64
dtype: object

Resumen de Estadísticos Descriptivos (Cuantitativos):
                     count          mean           std  ...       50%       75%       max
metrics.cpu_usage_pct  300.0     51.684100     26.465213  ...    49.845     74.71     98.88
metrics.latency_ms     300.0    221.386667    122.321047  ...    220.500    323.25    450.00
metrics.bytes_sent     300.0  32430.523333  19141.536076  ...  30721.500  48923.50  65309.00

[3 rows x 8 columns]
------------------------------------------------------------
```

Figura 3. Salida del script: perfilamiento de raw/logs_m2m.json.

Interpretación. Un JSON no es tabular, por lo que se usó `pd.json_normalize`, que "aplana" el objeto anidado `metrics` en columnas con notación de punto. El resultado fue un DataFrame de **300 filas por 9 columnas**: seis campos de texto (`log_id`, `timestamp`, `device_id`, `ip_address`, `level`, `event`) y tres numéricos que provienen de `metrics`. Los 300 registros coinciden con los generados y subidos en el paso anterior.

Columna	Tipo	count	mean	std	50%	75%	max
metrics.cpu_usage_pct	float64	300	51.68	26.47	49.85	74.71	98.88
metrics.latency_ms	int64	300	221.39	122.32	220.50	323.25	450
metrics.bytes_sent	int64	300	32,430.52	19,141.54	30,721.50	48,923.25	65,309

Tabla 5. Estadísticos descriptivos de las métricas del JSON (columnas visibles en la salida de consola).

Las tres métricas tienen 300 valores, por lo que no hay nulos. En todas la media y la mediana están muy próximas (51.68 frente a 49.85 de uso de CPU, 221.39 frente a 220.50 ms de latencia y 32,430 frente a 30,721 bytes enviados), lo que indica distribuciones balanceadas. Los valores máximos (98.88 %, 450 ms y 65,309 bytes) se encuentran dentro de los rangos con los que se generaron los datos, y la latencia llega justo al tope de 450 ms. Cabe señalar que la consola recortó la tabla de `describe()` (se ven puntos suspensivos y la leyenda "[3 rows x 8 columns]"), por lo que las columnas *min* y *25%* no aparecen en la captura; este detalle se aborda en la sección 3.3.

2.4 Verificación en la consola de AWS S3

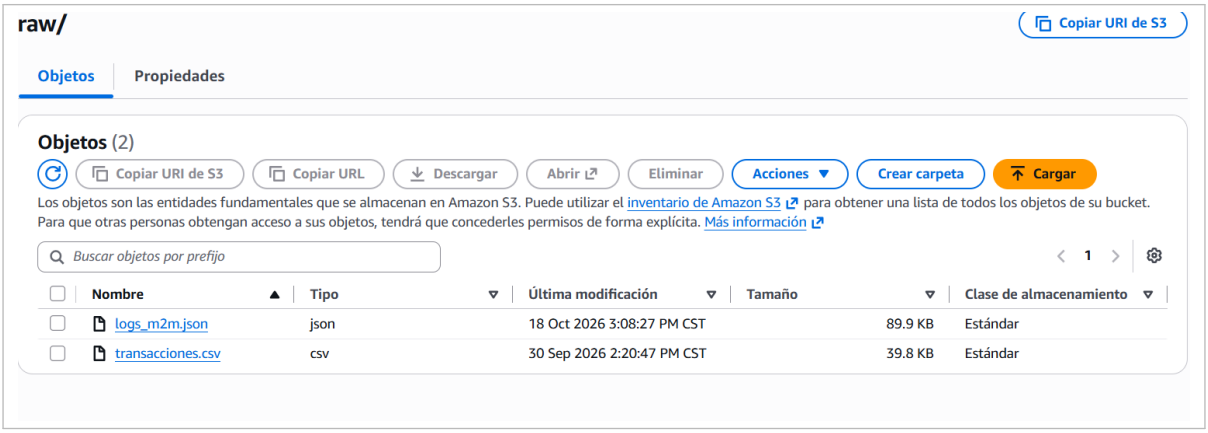


Figura 4. Contenido del prefijo raw/ en la consola de Amazon S3.

La consola confirma que el prefijo `raw/` contiene los dos objetos esperados. El archivo `logs_m2m.json` (89.9 KB) tiene fecha de modificación del 18 de octubre de 2026, es decir, el día de la ejecución, lo que demuestra que fue generado y cargado por el script, mientras que `transacciones.csv` (39.8 KB) ya existía desde el 30 de septiembre de 2026.

3. Análisis de Cambios y Ajustes

3.1 Ajustes realizados al código base

Elemento	Código base	Ajuste realizado	Motivo
BUCKET_NAME	'aws-academy-datacenter-bucket'	'aws-academy-datacenter-bucket-23-02-004'	Nombre real del bucket asignado en el entorno.
Credenciales	boto3.client sin claves	Claves temporales como parámetros en la ejecución; en Git, sin claves	La sesión de AWS Academy exige token temporal; el repositorio no debe contener secretos.
Objeto JSON	Se asume que raw/logs_m2m.json existe	Nueva función generar_y_subir_logs_json()	El objeto no existía en el bucket; sin él, get_object fallaría.
Manejo de errores	Sin try/except	try/except con NoSuchKey y Exception; la función devuelve None si falla	Evitar que el pipeline se detenga con un error no controlado y dar mensajes claros.
Mensajes de salida	Solo perfilamiento	Mensajes de progreso y separadores	Facilitar la lectura e interpretación de la ejecución.

Tabla 6. Resumen de cambios respecto al fragmento base de la práctica.

3.2 Resolución de errores y manejo de excepciones

El fragmento base llama a `get_object` sin protección, de modo que un objeto inexistente produce una excepción `NoSuchKey` que detiene todo el programa. Para resolverlo se hicieron dos cosas. Primero, se agregó la función `generar_y_subir_logs_json`, que crea los 300 registros, los serializa con `json.dumps(...).encode("utf-8")` y los sube con `put_object`, de manera que el archivo JSON existe antes de perfilarlo. Segundo, se envolvió la ingesta en un bloque `try/except` que distingue dos casos: la clave no encontrada, que muestra el bucket y la clave consultados, y cualquier otro error (por ejemplo, credenciales vencidas, falta de permisos o un JSON mal formado), que imprime el mensaje de la excepción. En ambos casos la función devuelve `None` en lugar de interrumpir la ejecución.

```
try:
    response = s3_client.get_object(Bucket=BUCKET_NAME, Key=file_key)
    body = response['Body'].read()          # bytes en memoria
    if is_json:
        df = pd.json_normalize(json.loads(body.decode('utf-8')))
    else:
        df = pd.read_csv(io.BytesIO(body))   # sin guardar a disco
    ...
    return df
except s3_client.exceptions.NoSuchKey:
    print(f"ERROR: El archivo '{file_key}' no fue encontrado en el bucket '{BUCKET_NAME}'.")
    return None
except Exception as e:
    print(f"ERROR al procesar {file_key}: {str(e)}")
    return None
```

Una consideración adicional es que, aunque la semilla aleatoria es fija, las fechas de los logs se calculan a partir de `datetime.now()`, por lo que cada ejecución vuelve a crear el archivo con marcas de tiempo distintas y sobrescribe el objeto anterior en S3. El tamaño y la estructura se mantienen, pero los valores de `timestamp` cambian.

3.3 Inconsistencias detectadas y mejoras propuestas

Al interpretar los resultados se identificaron tres puntos que el perfilamiento básico no resuelve por sí solo, y que se proponen como adecuaciones al pipeline:

Hallazgo	Evidencia	Mejora propuesta
Fechas como texto	fecha (CSV) y timestamp (JSON) aparecen como str	pd.to_datetime(df['fecha'], errors='coerce'); los valores no convertibles quedan como NaT y se pueden contabilizar.
Nulos en columnas de texto	describe() solo cubre columnas numéricas; el count de monto = 500 descarta nulos solo ahí	Agregar df.isna().sum() para revisar los nulos de todas las columnas.
Salida truncada	La tabla del JSON se muestra con "..." y "[3 rows x 8 columns]"	pd.set_option('display.max_columns', None) para ver min, 25% y demás estadísticos.

Tabla 7. Hallazgos de calidad de datos y ajustes sugeridos.

```
df['fecha'] = pd.to_datetime(df['fecha'], errors='coerce')
print(df.isna().sum())                       # nulos por columna
pd.set_option('display.max_columns', None)   # evitar el recorte en consola
```

4. Conclusiones

4.1 Ingesta directa en memoria (io.BytesIO) frente a descarga previa en disco

Aspecto	Directo a memoria (BytesIO)	Descarga previa a disco
Velocidad	Evita escribir y volver a leer el archivo; menos operaciones de E/S.	Agrega una escritura y una lectura en disco antes de procesar.
Seguridad y limpieza	No quedan copias de los datos en el equipo ni archivos temporales por borrar.	Deja copias locales que hay que proteger y eliminar.
Consumo de memoria	read() carga todo el objeto en RAM; se vuelve un límite con archivos muy grandes.	Permite procesar por partes sin cargar todo en RAM.
Reutilización	Cada ejecución vuelve a descargar el objeto desde S3.	La copia local se reutiliza sin consumir red.
Entornos sin disco persistente	Funciona bien en contenedores y funciones serverless.	Depende del espacio temporal disponible (p. ej. /tmp).

Tabla 8. Comparación de las dos estrategias de ingesta.

Para los archivos de esta práctica (39.8 KB y 89.9 KB) la ingesta directa desde memoria es claramente la mejor opción: es más simple, no deja rastros en el disco y evita gestionar archivos temporales. Su límite está en el tamaño, porque `read()` trae el objeto completo a la memoria RAM; con archivos de varios gigabytes sería preferible leer por bloques (por ejemplo con el parámetro `chunksize` de `read_csv`), usar formatos columnares como Parquet o descargar el archivo a disco.

4.2 Conclusiones generales

Se logró conectar un script de Python a S3 con boto3, ingerir un archivo estructurado y uno semiestructurado, y perfilarlos con pandas: el CSV de transacciones (500 filas, 7 columnas, con `monto` como única variable numérica) y el JSON de logs M2M (300 filas, 9 columnas tras aplanar el objeto anidado `metrics`). Esto mostró que `pd.json_normalize` permite tratar datos semiestructurados con las mismas herramientas de análisis que los tabulares.

El manejo de excepciones y la creación del objeto faltante hicieron que el pipeline fuera más robusto, y el trabajo con credenciales temporales dejó claro que los secretos nunca deben escribirse en el código que se sube a Git, sino leerse del entorno o de `~/.aws/credentials` y excluirse con `.gitignore`. Por último, el perfilamiento reveló que las fechas llegan como texto y que conviene revisar nulos en todas las columnas, ajustes que serían el siguiente paso natural para llevar estos datos de la zona *raw* a una capa limpia y lista para análisis.